

Blatt 9

Software Qualität

Unit-Testing mit JUnit

Luis Staudt

Aufgabe 1: Unit-Test für TemperatureConverter

Die Klasse `TemperatureConverter` besitzt zwei reine (zustandslose) Methoden zur Umrechnung zwischen Celsius und Fahrenheit. Da keine Verzweigungen oder Zustände existieren, genügen für jede Methode einige repräsentative Eingabewerte (typische Werte, Gefrier-/Siedepunkt, negative Werte, Hin- und Rückrechnung).

Getestete Klasse

```
1 public class TemperatureConverter {  
2  
3     // Konvertiert Celsius in Fahrenheit  
4     public double celsiusToFahrenheit(double celsius) {  
5         return (celsius * 9 / 5) + 32;  
6     }  
7  
8     // Konvertiert Fahrenheit in Celsius  
9     public double fahrenheitToCelsius(double fahrenheit) {  
10        return (fahrenheit - 32) * 5 / 9;  
11    }  
12 }
```

Testfälle

Nr.	Methode / Eingabe	Erwartung	Zweck
T1	celsiusToFahrenheit(0)	32,0	Gefrierpunkt
T2	celsiusToFahrenheit(100)	212,0	Siedepunkt
T3	celsiusToFahrenheit(−40)	−40,0	Schnittpunkt der Skalen
T4	fahrenheitToCelsius(32)	0,0	Gefrierpunkt
T5	fahrenheitToCelsius(212)	100,0	Siedepunkt
T6	fahrenheitToCelsius(−40)	−40,0	Schnittpunkt der Skalen
T7	Hin- und Rückrechnung	Ausgangswert	Konsistenz

Hinweis: Da Gleitkommazahlen verglichen werden, wird die Überladung `assertEquals(expected, actual, delta)` mit einer kleinen Toleranz ($\delta = 0,0001$) verwendet.

Testklasse

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class TemperatureConverterTest {
5
6     private final TemperatureConverter converter = new TemperatureConverter();
7     private static final double DELTA = 0.0001;
8
9     @Test
10    void celsiusToFahrenheit_Gefrierpunkt() {
11        assertEquals(32.0, converter.celsiusToFahrenheit(0), DELTA);
12    }
13
14    @Test
15    void celsiusToFahrenheit_Siedepunkt() {
16        assertEquals(212.0, converter.celsiusToFahrenheit(100), DELTA);
17    }
18
19    @Test
20    void celsiusToFahrenheit_negativ() {
21        assertEquals(-40.0, converter.celsiusToFahrenheit(-40), DELTA);
22    }
23
24    @Test
25    void fahrenheitToCelsius_Gefrierpunkt() {
26        assertEquals(0.0, converter.fahrenheitToCelsius(32), DELTA);
27    }
28
29    @Test
30    void fahrenheitToCelsius_Siedepunkt() {
31        assertEquals(100.0, converter.fahrenheitToCelsius(212), DELTA);
32    }
33
34    @Test
35    void fahrenheitToCelsius_negativ() {
36        assertEquals(-40.0, converter.fahrenheitToCelsius(-40), DELTA);
37    }
38
39    @Test
40    void hinUndRueckrechnungIstKonsistent() {
41        double celsius = 37.5;
42        double fahrenheit = converter.celsiusToFahrenheit(celsius);
43        assertEquals(celsius, converter.fahrenheitToCelsius(fahrenheit), DELTA);
44    }
45 }
```

Aufgabe 2: Unit-Test für BankAccount

Die Klasse `BankAccount` verwaltet einen Kontostand mit den Operationen `deposit` (Einzahlen) und `withdraw` (Abheben). Beide Methoden enthalten Validierungen, die bei ungültigen Beträgen eine `IllegalArgumentException` werfen. Getestet werden daher sowohl die gültigen (Happy-Path) als auch die ungültigen Pfade (Ausnahmen).

Getestete Klasse

```
1 public class BankAccount {
2     private double balance;
3
4     public BankAccount() {
5         this.balance = 0.0;
6     }
7
8     public double getBalance() {
9         return balance;
10    }
11
12    public void deposit(double amount) {
13        if (amount <= 0) {
14            throw new IllegalArgumentException("Deposit amount must be positive");
15        }
16        balance += amount;
17    }
18
19    public void withdraw(double amount) {
20        if (amount <= 0) {
21            throw new IllegalArgumentException("Withdrawal amount must be positive");
22        }
23        if (amount > balance) {
24            throw new IllegalArgumentException("Insufficient funds");
25        }
26        balance -= amount;
27    }
28 }
```

Testfälle

Nr.	Szenario	Erwartetes Verhalten
T1	Neues Konto	Kontostand = 0,0
T2	deposit(100)	Kontostand = 100,0
T3	deposit(0) bzw. deposit(-50)	IllegalArgumentException
T4	deposit(100), withdraw(40)	Kontostand = 60,0
T5	withdraw(0) bzw. withdraw(-10)	IllegalArgumentException
T6	withdraw(50) bei Kontostand 0	IllegalArgumentException (Insufficient funds)
T7	withdraw genau in Höhe des Kontostands	Kontostand = 0,0 (Grenzfall)

Testklasse

```

1 import org.junit.jupiter.api.BeforeEach;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 public class BankAccountTest {
6
7     private BankAccount account;
8     private static final double DELTA = 0.0001;
9
10    @BeforeEach
11    void setUp() {
12        account = new BankAccount();
13    }
14
15    @Test
16    void neuesKontoHatGuthabenNull() {
17        assertEquals(0.0, account.getBalance(), DELTA);
18    }
19
20    @Test
21    void depositErhoehtGuthaben() {
22        account.deposit(100);
23        assertEquals(100.0, account.getBalance(), DELTA);
24    }
25
26    @Test
27    void depositMitNichtPositivemBetragWirftException() {
28        assertThrows(IllegalArgumentException.class, () -> account.deposit(0));
29        assertThrows(IllegalArgumentException.class, () -> account.deposit(-50));
30    }
31
32    @Test
33    void withdrawVerringertGuthaben() {
34        account.deposit(100);
35        account.withdraw(40);

```

```
36     assertEquals(60.0, account.getBalance(), DELTA);
37 }
38
39 @Test
40 void withdrawMitNichtPositivemBetragWirftException() {
41     assertThrows(IllegalArgumentException.class, () -> account.withdraw(0));
42     assertThrows(IllegalArgumentException.class, () -> account.withdraw(-10));
43 }
44
45 @Test
46 void withdrawUeberGuthabenWirftException() {
47     Exception ex = assertThrows(IllegalArgumentException.class, () -> account.withdraw(50)
48 );
49     assertEquals("Insufficient funds", ex.getMessage());
50 }
51
52 @Test
53 void withdrawGesamtesGuthabenErlaubt() {
54     account.deposit(50);
55     account.withdraw(50);
56     assertEquals(0.0, account.getBalance(), DELTA);
57 }
```

Aufgabe 3: ShoppingCart

Es wird eine Klasse `ShoppingCart` implementiert, die das Hinzufügen und Entfernen von Artikeln (Name + Preis) sowie die Berechnung des Gesamtpreises ermöglicht. Anschließend wird die Funktionalität mit JUnit getestet.

Klasse Item

```
1 public class Item {
2     private final String name;
3     private final double price;
4
5     public Item(String name, double price) {
6         this.name = name;
7         this.price = price;
8     }
9
10    public String getName() { return name; }
11    public double getPrice() { return price; }
12 }
```

Klasse ShoppingCart

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class ShoppingCart {
5     private final List<Item> items = new ArrayList<>();
6
7     // Artikel mit Namen und Preis hinzufuegen
8     public void addItem(String name, double price) {
9         items.add(new Item(name, price));
10    }
11
12    // Ersten Artikel mit dem angegebenen Namen entfernen
13    public boolean removeItem(String name) {
14        return items.removeIf(item -> item.getName().equals(name));
15    }
16
17    // Gesamtpreis aller Artikel berechnen
18    public double getTotalPrice() {
19        double total = 0.0;
20        for (Item item : items) {
21            total += item.getPrice();
22        }
23    }
24 }
```

```
22     }
23     return total;
24 }
25
26 public int getItemCount() {
27     return items.size();
28 }
29 }
```

Testklasse

```
1 import org.junit.jupiter.api.BeforeEach;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 public class ShoppingCartTest {
6
7     private ShoppingCart cart;
8     private static final double DELTA = 0.0001;
9
10    @BeforeEach
11    void setUp() {
12        cart = new ShoppingCart();
13    }
14
15    @Test
16    void neuerWarenkorbIstLeer() {
17        assertEquals(0, cart.getItemCount());
18        assertEquals(0.0, cart.getTotalPrice(), DELTA);
19    }
20
21    @Test
22    void addItemFuegtArtikelHinzu() {
23        cart.addItem("Apfel", 0.50);
24        assertEquals(1, cart.getItemCount());
25        assertEquals(0.50, cart.getTotalPrice(), DELTA);
26    }
27
28    @Test
29    void getTotalPriceSummiertAlleArtikel() {
30        cart.addItem("Apfel", 0.50);
31        cart.addItem("Brot", 2.20);
32        cart.addItem("Milch", 1.30);
33        assertEquals(4.00, cart.getTotalPrice(), DELTA);
34    }
35
36    @Test
37    void removeItemEntferntArtikel() {
38        cart.addItem("Apfel", 0.50);
39        cart.addItem("Brot", 2.20);
40        boolean removed = cart.removeItem("Apfel");
41        assertTrue(removed);
42        assertEquals(1, cart.getItemCount());
43        assertEquals(2.20, cart.getTotalPrice(), DELTA);
44    }
45 }
```



```
46  @Test
47  void removeItemMitUnbekanntemNamenAendertNichts() {
48      cart.addItem("Apfel", 0.50);
49      boolean removed = cart.removeItem("Banane");
50      assertFalse(removed);
51      assertEquals(1, cart.getItemCount());
52  }
53 }
```

Aufgabe 4: Erweiterter ShoppingCart mit Stub

Der Preis eines Artikels wird nun nicht mehr direkt übergeben, sondern von der Klasse `ArtikelPreis` bezogen (z. B. aus einer Datenbank). Um den `ShoppingCart` unabhängig von einer realen Datenbankverbindung testen zu können, wird in der Testklasse ein **Stub** für `ArtikelPreis` erstellt. Der Stub liefert fest verdrahtete Preise und ersetzt so die echte Datenquelle.

Klasse `ArtikelPreis` (Produktivcode mit Datenbank)

```
1 import java.util.HashMap;
2 import java.util.Map;
3
4 public class ArtikelPreis {
5     // simuliert eine lokale Datenbank/Tabelle
6     private final Map<String, Double> preise = new HashMap<>();
7
8     public ArtikelPreis() {
9         preise.put("Apfel", 0.50);
10        preise.put("Brot", 2.20);
11        preise.put("Milch", 1.30);
12    }
13
14    // Preis fuer einen gegebenen Artikel abfragen
15    public double getPreis(String artikelName) {
16        Double preis = preise.get(artikelName);
17        if (preis == null) {
18            throw new IllegalArgumentException("Unbekannter Artikel: " + artikelName);
19        }
20        return preis;
21    }
22 }
```

Damit der `ShoppingCart` im Test gegen einen Stub austauschbar ist, wird eine gemeinsame Schnittstelle eingeführt und über den Konstruktor injiziert (*Dependency Injection*).

```
1 public interface IArtikelPreis {
2     double getPreis(String artikelName);
3 }
```

(Die Klasse `ArtikelPreis` implementiert `IArtikelPreis`, also `public class ArtikelPreis implements IArtikelPreis`.)

Klasse ShoppingCart (erweitert)

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class ShoppingCart2 {
5     private final List<Item> items = new ArrayList<>();
6     private final IArtikelPreis artikelPreis;
7
8     public ShoppingCart2(IArtikelPreis artikelPreis) {
9         this.artikelPreis = artikelPreis;
10    }
11
12    // Preis wird ueber ArtikelPreis bezogen
13    public void addItem(String name) {
14        double price = artikelPreis.getPreis(name);
15        items.add(new Item(name, price));
16    }
17
18    public boolean removeItem(String name) {
19        return items.removeIf(item -> item.getName().equals(name));
20    }
21
22    public double getTotalPrice() {
23        double total = 0.0;
24        for (Item item : items) {
25            total += item.getPrice();
26        }
27        return total;
28    }
29
30    public int getItemCount() {
31        return items.size();
32    }
33 }

```

Testklasse mit Stub

Der Stub implementiert dieselbe Schnittstelle, liefert aber feste Preise ohne Datenbankzugriff. So lässt sich der **ShoppingCart** isoliert testen.

```

1 import org.junit.jupiter.api.BeforeEach;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 public class ShoppingCartStubTest {
6
7     private static final double DELTA = 0.0001;
8
9     // Stub-Klasse: ersetzt die reale Datenbankanbindung
10    static class ArtikelPreisStub implements IArtikelPreis {
11        @Override
12        public double getPreis(String artikelName) {
13            switch (artikelName) {
14                case "Apfel": return 1.00;

```

```
15         case "Brot": return 3.00;
16         default: throw new IllegalArgumentException("Unbekannt: " + artikelName);
17     }
18 }
19 }
20
21 private ShoppingCart cart;
22
23 @BeforeEach
24 void setUp() {
25     cart = new ShoppingCart(new ArtikelPreisStub());
26 }
27
28 @Test
29 void addItemBeziehtPreisVomStub() {
30     cart.addItem("Apfel");
31     assertEquals(1.00, cart.getTotalPrice(), DELTA);
32 }
33
34 @Test
35 void getTotalPriceMitMehrerenArtikeln() {
36     cart.addItem("Apfel");
37     cart.addItem("Brot");
38     assertEquals(4.00, cart.getTotalPrice(), DELTA);
39 }
40
41 @Test
42 void removeItemEntferntArtikel() {
43     cart.addItem("Apfel");
44     cart.addItem("Brot");
45     cart.removeItem("Apfel");
46     assertEquals(3.00, cart.getTotalPrice(), DELTA);
47     assertEquals(1, cart.getItemCount());
48 }
49
50 @Test
51 void unbekannterArtikelWirftException() {
52     assertThrows(IllegalArgumentException.class, () -> cart.addItem("Banane"));
53 }
54 }
```

Vorteil des Stubs: Der Test ist schnell, deterministisch und unabhängig von der Verfügbarkeit oder dem Inhalt der realen Datenbank. Es wird ausschließlich die Logik des ShoppingCart geprüft, nicht die der Datenquelle.